

Program:

A program is a set of instructions developed to perform a task. Each instruction acts as a command for the computer. A program is developed using a programming language.

Characteristics of a good program:

The language used to develop a program should always be clear and readable. Every program has a structure. If the structure is well-defined, the program can be easily understood and modified. A program which can be understood can also be easily analyzed and tested for correctness. Any program that requires debugging is easier to debug if the creator knows what is going on. The following are some of the benefits of good programming practices:

1. Easy to code
2. Easy to debug
3. Easy to test
4. Reliability
5. Maintainability
6. Portability
7. User friendly

Concept of Modular Programming:

Modular programming is the process of subdividing a program into separate subprograms such as functions and subroutines. Subprograms make the actual program shorter, hence easier to read and understand. Further, the arguments show exactly what information a subprogram is using. This makes it easier to figure out whether it needs to be changed or not when the program is being modified. Modular approach makes debugging, testing and maintenance very easy and smooth.

Some of the advantages of using modular approach are:

1. Easy to read
2. Easy to understand
3. Easy to debug
4. Easy to test
5. Easy to modify

Program Documentation:

Documentation is a written detailed description of the programming cycle and specific facts about the program. Using these, the programmer can easily recall actions to be performed during the testing of a program while modifying, maintaining and enhancing the program.

Documentation can be divided into two types:

A. External Documentation: The external documentation of a program includes specifications, development history and design with flow of control. It is also known as user documentation. It provides information to the user such as:

- 1) Description and purpose of the program.
- 2) Description of procedures to start the program.
- 3) Information regarding the input data.
- 4) Expected output of the program.
- 5) The information about the messages likely to be displayed while execution of the Program.
- 6) Drawbacks of the program.

B. Internal Documentation: It is required for programmers who desire to modify the program for the purpose of program maintenance. This program documentation consists of:

- 1) Use of meaningful variable names.
- 2) A written description of the program.
- 3) Purpose of the program.
- 4) Purpose of various sub-routines and functions.
- 5) Comments to improve the clarity of the program.
- 6) Statements to begin on a separate line.
- 7) A blank line is used before a function or a subroutine.
- 8) Adding explanatory notes to variables.
- 9) Clarifying conditions used by the program

Program Maintenance:

The process of updating or providing new versions of a program so that the program meets present day requirements of the user is often referred to as program maintenance. Program maintenance may be required for the following reasons:

1. New errors or bugs, which are not detected during development and testing, can be detected during the usage of the program.
2. The needs of the user have changed over a period of time and the program has to be modified to meet the present day needs.
3. The existing program is not functioning to the satisfaction of the user, so the program has to be modified to provide the additional information.
4. The user has purchased a new computer or hardware and the program is not functioning properly on the new system.
5. The user has seen the features of new type of software and thus feels that his existing system has to be changed to a new system.

Types of Program maintenance:

1. **Corrective maintenance:** It is required to fix errors that are found after delivery of the software.
2. **Adaptive maintenance:** This type of maintenance deals with the modification of a software to meet the changing needs of the environment.

3. **Perfective maintenance:** It is used to improve performance of the software by adding new capabilities and facilities to adapt new technologies.
4. **Preventive maintenance:** This type of maintenance deals with correcting the errors or faults in the software before they become effective faults.

Debugging:

Debugging is the process of finding and removing errors in the software code. An error is a mistake or an unexpected result that is detected while compiling or running a program. Bug is a common name for a software error. Debugging is a very important and time-consuming phase of software development. Software error can be a syntax error, run time error or logical error.

Error and its types:

An error is a mistake or an unexpected result that is detected while compiling or running a program. The various types of errors are as follows.

1. **Syntax error:** A syntax error refers to the mistake in the syntax of a statement. In compiled programming languages, a syntax error is a fatal error that the programmer must correct before compiling the complete program. Syntax error may occur due to reasons like entering an invalid equation for calculation, by missing a semicolon, by forgetting a closing bracket, or using several decimal points in one number etc.
2. **Run-time error:** Run-time errors are those that appear while executing the code. The code will have no syntax errors, but will not be executed.
Runtime errors can be caused by a variety of factors, such as dividing by zero, accessing a non-existent memory location, or trying to use an undefined variable. They are often the result of unexpected conditions or user inputs that the programmer did not account for.
Examples of Runtime Errors:
 - . Division by zero.
 - . File not found etc.
3. **Logical error:** Logical errors occur when desired or expected results are not obtained during program execution due to bugs in code. With logical errors, the program runs, but produces incorrect results. For example, the programmer has used a division operator instead of multiplication operator in a subroutine developed for income tax calculation and hence gives incorrect output. Logical errors are generally the hardest type to fix, since it is not always clear where they originate.

Algorithm:

Algorithm is a step-by-step procedure to solve a problem. Algorithm can be represented using pseudo code. Pseudo code is the outline of a program, written in a form of plain English text that can be easily converted into a high-level program. They are not specific to any programming language.

Characteristics of an algorithm:

The essential characteristics of an algorithm are:

1. Every step of an algorithm should perform a single task.
2. Confusion should not be included at any stage in an algorithm.
3. An algorithm should involve a finite number of steps to arrive at a solution.
4. Simple statement and structure should be used in the development of the algorithm.
5. Every algorithm should lead to a unique solution of the problem.
6. An algorithm should have the capability to handle some unexpected situations, which may arise during the solution of a problem.

Flowchart:

A flowchart is a pictorial representation of an algorithm. A flowchart presents two aspects of a program's logic:

1. The operations required to solve a problem.
2. The sequence in which these operations are to be performed.

This is done by using a set of symbols, each representing specific operation that can be executed.

Use of different symbols in developing Flowcharts:

Terminal: Indicates the beginning and end of a program.



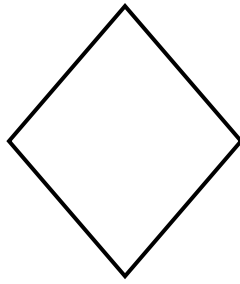
Input/Output: Inputs or outputs data or information.



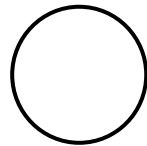
Processing: Represents calculations, data manipulations or information processing.



Decision: Represents a comparison, a question or a decision that determines alternative path to be followed.



Connector: Represents an entry from, or an exit to, another part of the program flowchart.



Flow Lines: Indicate the direction of processing or data flow

